



STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Dr. Jayaram Boga
Student Name	Dharmika. P
Roll Number	2520030012
Branch	CSE
Course Outcome	CO-1 Apply balanced tree data structures and algorithms to efficiently store, manage, and retrieve data in real-world applications.

Case Study Topic: Netflix Recently Watched Index – AVL Tree

Netflix maintains a per-user Recently Watched Index to store the timestamps of movies and series watched by each user. The index is organized by watch timestamp in descending order, where the most recent watch activity appears first.

Each time a user watches content, a new timestamp is inserted into the index. Since users frequently open the application to view their watch history, the system must quickly retrieve the most recent items while also supporting continuous insertions throughout the day.

To ensure efficient insertion, search, and retrieval operations, Netflix uses an AVL Tree, a self-balancing binary search tree. Larger timestamps are stored in the left subtree so that an in-order traversal naturally returns the watch history from most recent to oldest.

Work Done Summary:

In this case study, an AVL Tree was constructed by inserting eleven watch timestamps in arrival order. During insertion, the balance factor of each node was monitored and AVL rotations were performed whenever the tree became unbalanced. After all insertions, the final balanced AVL Tree was obtained. An in-order traversal method was then implemented to retrieve the top five most recent timestamps from the tree. The performance of the AVL Tree was also compared with a sorted ArrayList to evaluate its suitability for large-scale streaming workloads. Finally, the time complexity of insertion, searching, and top-k retrieval operations was analyzed.

Implementation Details:

Input Timestamps

31000, 27000, 35000, 23000, 39000, 19000, 43000, 15000, 47000, 11000, 51000

Operations Performed

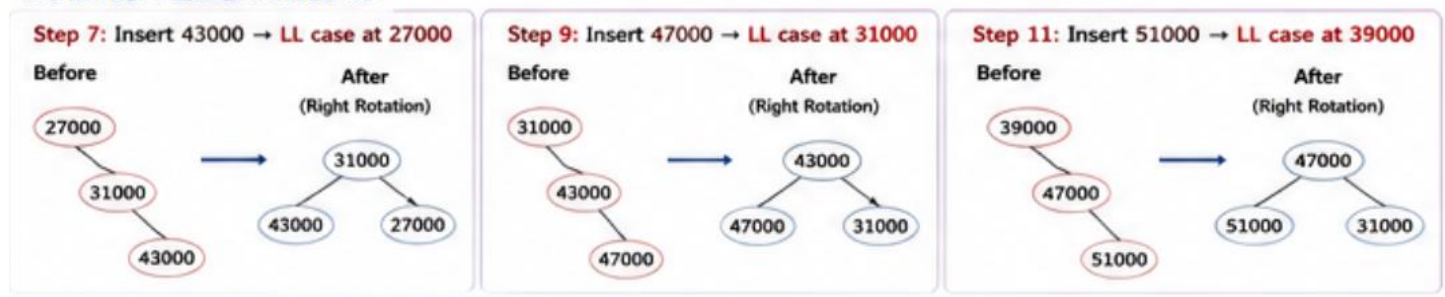
1. Inserted timestamps into AVL Tree.
2. Maintained descending timestamp ordering.
3. Applied AVL rotations whenever imbalance occurred.
4. Calculated balance factors.
5. Performed Top-K descending traversal.
6. Retrieved top 5 most recent timestamps.

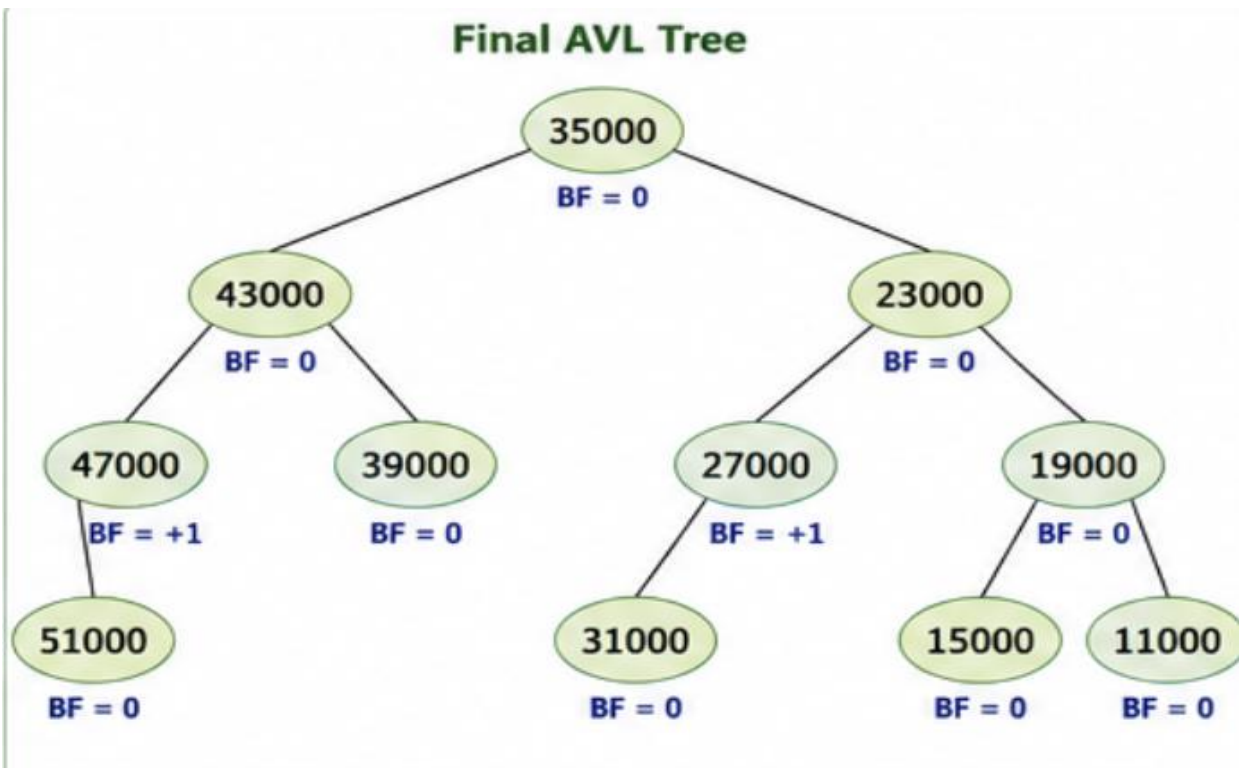
A) Build the AVL tree by inserting all 11 timestamps in arrival order (the tree is keyed descending — larger timestamp goes left). For every insertion that triggers rotation, identify the rotation type (LL / RR / LR / RL), the pivot, and the resulting balance factors of the affected subtree. Show the final AVL tree.

Solution:

Step	Insert	Rotation	Pivot	Why Rotation Happened	Balance of Affected Subtree (after rotation)
1	21000	—	—	Tree was empty	—
2	27000	—	—	Balanced	—
3	35000	—	—	Balanced	—
4	23000	—	—	Balanced	—
5	39000	—	—	Balanced	—
6	19000	—	—	Balanced	—
7	43000	LL	27000	Left subtree of 27000 became left heavy	BF(27000) = 0, BF(31000) = 0
8	15000	—	—	Balanced	—
9	47000	LL	31000	Left subtree of 31000 became left heavy	BF(31000) = 0, BF(43000) = +1
10	11000	—	—	Balanced	—
11	51000	LL	39000	Left subtree of 39000 became left heavy	BF(39000) = 0, BF(47000) = 0

ROTATION EXPLANATIONS





Balance Factors of Final Tree:

35000 → 0

43000 → 0

23000 → 0

47000 → +1

39000 → 0

27000 → +1

19000 → 0

51000 → 0

15000 → 0

11000 → 0

31000 → 0

B) Complete the Java boilerplate below. The AVLNode class and helpers are given; you fill the topKDescending method that performs an in-order traversal pruned at k elements (since the tree is keyed descending, in-order naturally yields most-recent first). The early-termination matters because $k = 50 \ll n$ at scale.

Apply your function (by hand) to the tree from (a) with $k = 5$. List the 5 timestamps returned. State the worst-case time complexity in terms of n (tree size) and k .

Solution:

Result: Screenshots / Output

```
import java.util.*;

class AVLNode {
    long ts;
    int height;
    AVLNode left, right;

    AVLNode(long ts) {
        this.ts = ts;
        this.height = 1;
    }
}

public class AVLTree {

    static int height(AVLNode node) {
        return (node == null) ? 0 : node.height;
    }

    static int getBalance(AVLNode node) {
        return (node == null) ? 0 :
            height(node.left) - height(node.right);
    }

    static AVLNode rightRotate(AVLNode y) {
        AVLNode x = y.left;
        AVLNode T2 = x.right;

        x.right = y;
        y.left = T2;

        y.height = Math.max(height(y.left),
            height(y.right)) + 1;
    }
}
```

```
    x.height = Math.max(height(x.left),height(x.right)) + 1;

    return x;
}
```

```
static AVLNode leftRotate(AVLNode x) {
    AVLNode y = x.right;
    AVLNode T2 = y.left;

    y.left = x;
    x.right = T2;

    x.height = Math.max(height(x.left),height(x.right)) + 1;

    y.height = Math.max(height(y.left), height(y.right)) + 1;

    return y;
}
```

```
static AVLNode insert(AVLNode root, long key) {
```

```
    if (root == null)
        return new AVLNode(key);

    // Larger timestamp goes LEFT
    if (key > root.ts)
        root.left = insert(root.left, key);
    else
        root.right = insert(root.right, key);
```



```
    root.height = 1 + Math.max(height(root.left),height(root.right));
```

```

static AVLNode insert(AVLNode root, long key) {
    int balance = getBalance(root);

    // LL Rotation
    if (balance > 1 && key > root.left.ts)
        return rightRotate(root);

    // RR Rotation
    if (balance < -1 && key < root.right.ts)
        return leftRotate(root);

    // LR Rotation
    if (balance > 1 && key < root.left.ts) {
        root.left = leftRotate(root.left);
        return rightRotate(root);
    }

    // RL Rotation
    if (balance < -1 && key > root.right.ts) {
        root.right = rightRotate(root.right);
        return leftRotate(root);
    }

    return root;
}

static void topK(AVLNode root, int k, List<Long> result) {
    if (root == null || result.size() == k)
        return;

    topK(root.left, k, result);

```

```

        if (result.size() < k)
            result.add(root.ts);

        topK(root.right, k, result);
    }

    static void inorder(AVLNode root) {
        if (root == null)
            return;

        inorder(root.left);
        System.out.print(root.ts + " ");
        inorder(root.right);
    }

```

Run | Debug

```

public static void main(String[] args) {

    long[] timestamps = {
        31000, 27000, 35000,
        23000, 39000, 19000,
        43000, 15000, 47000,
        11000, 51000
    };

    AVLNode root = null;

    for (long ts : timestamps) {
        root = insert(root, ts);
    }
}

```

```

    AVLNode root = null;

    for (long ts : timestamps) {
        root = insert(root, ts);
    }

    System.out.println(x: "All Timestamps (Descending Order):");
    inorder(root);

    List<Long> top5 = new ArrayList<>();
    topK(root, k: 5, top5);

    System.out.println(x: "\n\nTop 5 Recent Timestamps:");

    for (long ts : top5) {
        System.out.print(ts + " ");
    }
}

```

```

all\temp\vscode\644bd\jdt_ws\jdt.ls-java-project\bin\AVLTree
All Timestamps (Descending Order):
51000 47000 43000 39000 35000 31000 27000 23000 19000 15000 11000

Top 5 Recent Timestamps:
51000 47000 43000 39000 35000

```

Apply for $k = 5$

Since the tree is stored in descending order (largest values on the left), in-order traversal returns timestamps from most recent to oldest.

Returned Timestamps: 51000, 47000, 43000, 39000, 35000

Worst-Case Time Complexity

❖ Insertion : $O(\log n)$

❖ Search : $O(\log n)$

❖ Top-K Query : $O(\log n + k)$

❖ Space Complexity : $O(n)$

Reason:

- $O(\log n)$ to reach the first required node.
- $O(k)$ to collect k timestamps.
- AVL height remains $O(\log n)$.

C) Why AVL, not just a sorted ArrayList? An engineer suggests storing recently-played as `ArrayList<Long>` kept sorted descending: `Collections.binarySearch` for $O(\log n)$ lookup, `add()` at index for $O(\log n)$ search + $O(n)$ shift for insert. Argue quantitatively whether this is acceptable for Spotify's per-user workload. Use the production numbers: n grows to $\sim 10\,000$ plays over a year (no eviction); 30 inserts/day; 50 reads/day. Compute the ratio of total work between AVL and sorted-ArrayList over a one-year period. State the workload condition under which sorted-ArrayList wins.

Solution:

Workload (per year)

- n grows to $\sim 10,000$ plays
- 30 inserts/day, 50 reads/day $\rightarrow$ 10,950 inserts, 18,250 reads/year

Structure	Insert Cost	Read (Top-50) Cost	Total Work (approx.)
AVL Tree	$10,950 \times \log_2(10,000)$ $\approx 145,635$	$18,250 \times (\log_2(10,000) + 50)$ $\approx 1,155,225$	$\approx 1,300,860$
Sorted ArrayList	$10,950 \times 10,000$ $= 109,500,000$	$18,250 \times 50$ $= 912,500$	$\approx 110,412,500$

Work Ratio (ArrayList / AVL) = $110,412,500 / 1,300,860 \approx 84.9 \times$ more work
AVL Tree is ~ 85 times more efficient for this workload.

When Would Sorted ArrayList Win?

A sorted ArrayList is preferable when:

- Insertions are extremely rare.
- Reads are much more frequent than writes.
- Data size is small.
- Maintaining AVL balancing overhead is unnecessary.

GitHub Link: <https://github.com/Dharmika-12/DSA-2-Casestudy-1>

Student Signature	Faculty Signature